Unity Hostel Management System

Complete Step-by-Step Project Audit & Architecture Report

Project: Unity Hostel Management System	**Developer:** Aaditya (AadiCode04)
Stack: Node.js (v22+), Express, SQLite, HTML5/CSS3	**Repository:** github.com/Aaditya0411/Unity-Hostel_Management
Deployment Target: Render.com (Web Service)	**Report Date:** October 2026

1. Executive Overview (Project Ka Mukhya Uddeshya)

Unity Hostel Management System ek full-stack web application hai jo hostel administration aur students ke beech ke saare daily operations ko automate karta hai. Pehle jo kaam physical registers aur paperwork par hote the (jaise room allocation, attendance, fee collection, complaints aur notices), unhe ek unified, secure web portal mein convert kiya gaya hai.

Application ko **Node.js (Express)** backend aur modern responsive **HTML5/CSS3/JavaScript** frontend par build kiya gaya hai. Database ke liye modern **Node.js built-in SQLite (DatabaseSync)** engine use kiya gaya hai jisme kisi external heavy SQL database installation ki zaroorat nahi padti.

2. Kya-Kya Kiya Hai (Features Implemented - 100% Complete)

Project ke andar built aur tested saare modules ka step-by-step vivaran niche diya gaya hai:

Module / Feature	Status	Implementation Details (Kaise Kaam Karta Hai)
User Authentication & Security	DONE	• Student Signup with full name, email, password. • Password hashing using **bcryptjs** (salt rounds). • 6-digit OTP generation with SHA-256 hash storage & 10-minute expiry. • Secure Session via **JWT (JSON Web Token)** stored in HTTP-Only cookies. • Security headers implemented via **Helmet**.
Email & OTP Verification	DONE	• Nodemailer integration with SMTP credentials support. • **Console Fallback Mode:** Agar production SMTP configured na ho toh local terminal par OTP print hoti hai, jisse dev/testing kabhi block nahi hoti. • OTP Resend feature with rate limiting and timeout.
Password Reset Workflow	DONE	• Forgot Password form (email validation). • Email OTP delivery for identity verification. • Reset Password interface jisme OTP verify karke naya password bcrypt hash ke sath update hota hai.
Interactive Student Dashboard	DONE	• Overview analytics cards: Allocated Room No., Pending Fee Status, Active Complaints count, Unread Notices. • Student Profile details display. • Quick-access buttons for Room Booking, Fee Payment, Complaints filing.

Room Management & Allocation	**DONE**	• Room inventory: Single, Double, Triple sharing rooms. • Dynamic bed tracking: Total beds vs Available beds. • Floor allocation, Monthly rent, Amenities catalog. • Real-time booking logic jo bed availability validate karti hai.
Complaint & Maintenance Desk	**DONE**	• Issue filing categories: Plumbing, Electrical, Wi-Fi, Cleanliness, Food/Mess, Furniture. • Priority tagging (Low, Medium, High). • Status lifecycle tracking: *Pending* → *In Progress* → *Resolved* with admin remarks.
Fee Management & Ledger	**DONE**	• Automatic fee ledger creation per student. • Total fee, Paid amount, and Pending balance calculations. • Fee payment transaction logging with unique Transaction ID & timestamp. • Status auto-update to 'Paid' or 'Partial'.
Notice Board & Announcements	**DONE**	• Administrative circulars and event announcements. • Categorized notices (General, Hostel Meetings, Maintenance).
Attendance & Leave Requests	**DONE**	• Daily attendance data schema. • Student Outstation Leave application system (Dates, Reason, Status: Pending/Approved).
Contact & Helpdesk	**DONE**	• Public inquiry form for students/parents. • Inquiries directly stored in contact_messages database table.
Git & Cloud Deployment Setup	**DONE**	• Initialized clean Git repository with secure .gitignore. • Pushed code to GitHub (Aaditya0411/Unity-Hostel_Management). • Created render.yaml blueprint & configured Node.js >= 22.5.0 engines.

3. Kya Baki Hai / Kya Nahi Kiya Hai (Pending & Future Scope)

Exam/Viva ya production level perspective se project mein jo cheezein abhi implement nahi hui hain ya jinhe enhance kiya ja sakta hai:

Pending Area / Missing Feature	Priority	Description & Impact (Kyun Zaroori Hai)
Dedicated Admin / Warden UI Portal	**HIGH**	Backend mein complaints resolve karne aur notices add karne ke endpoints hain, lekin Warden ke liye alag se dedicated graphical UI (jaise admin-dashboard.html) nahi bana hai jahan se warden 1-click se complaints resolve ya room allocate kare.
Live Payment Gateway (Razorpay/Stripe)	**MEDIUM**	Abhi fee payment simulation mode mein hai (system dummy transaction ID generate karke ledger update karta hai). Real credit card/UPI payment ke liye Razorpay ya Stripe gateway SDK webhook ke sath link karna baki hai.
SMS Gateway Integration	**LOW**	Abhi OTP delivery sirf Email (SMTP) ya Console par aati hai. Mobile number par direct SMS OTP (Twilio ya Fast2SMS) integrate nahi hai.
Real-time WebSockets (Socket.IO)	**LOW**	Real-time notifications (jaise warden dwara complaint resolve karte hi student screen par popup aana) ke liye WebSocket live connection nahi hai; user ko page reload karna padta hai.
Render Ephemeral Storage Limitation	**MEDIUM**	Render Free Tier par local SQLite file container restart hone par reset ho sakti hai. Production mein permanent data ke liye Supabase PostgreSQL ya Render Persistent Disk attach karna recommend kiya jata hai.
Legacy Java Code Cleanup	**CLEANUP**	Project ke src/connection/ folder mein puraane Java Servlets aur DAO files hain. Kyunki ab pura server Node.js par chal raha hai, Java files unused hain aur unhe safely archive kiya ja sakta hai.

4. Database Architecture (10 Relational Tables)

Application SQLite ke andar 10 relational tables automatically manage karta hai:

Table Name	Columns & Keys	Business Logic / Purpose
users	id, full_name, email, password_hash, otp_hash, phone, room_id, college	Student profile, login credentials, and verification status
rooms	id, room_number, room_type, floor, total_beds, available_beds, monthly_fee	Hostel rooms catalog with live bed count and pricing
room_bookings	id, user_id (FK), room_id (FK), bed_number, booking_date, status	Record of confirmed student room allocations
fees	id, user_id (FK), total_amount, paid_amount, pending_amount, status	Summary of hostel dues and payment balance per student
fee_transactions	id, user_id (FK), amount, payment_method, transaction_id, notes	Audit history of individual fee payments with receipt IDs
complaints	id, user_id (FK), room_number, complaint_type, priority, description, status	Maintenance ticket tracking with status workflow
notices	id, title, category, description, author, created_at	Administrative circulars published for students
attendance	id, user_id (FK), date, status (Present/Absent), remarks	Daily attendance log for hostel safety and curfew
leave_requests	id, user_id (FK), leave_type, start_date, end_date, reason, status	Student outstation night-out and vacation pass requests
contact_messages	id, name, email, phone, subject, message, created_at	Public feedback and admission inquiries from contact page

5. Step-by-Step Render Deployment Guide

Application ko Render (https://render.com) par deploy karne ke liye niche diye steps follow karein:

Step 1: Render Dashboard par jaakar **'New +'** → **'Web Service'** par click karein.

Step 2: Apne GitHub repository **'Unity-Hostel_Management'** ko connect karein.

Step 3: Settings configure karein: *Runtime:* Node, *Build Command:* npm install, *Start Command:* npm start.

Step 4: Environment Variables (Critical ■■):
- NODE_VERSION = 22 (Zaroori hai kyunki project Node 22 ka built-in SQLite module use karta hai).
- JWT_SECRET = unityhostel-super-secret-key-32chars-long.

Step 5: 'Deploy Web Service' click karein. 1-2 minutes mein application globally live ho jaayegi!

6. Project Viva / Interview Important Questions & Answers

Q1: Backend kis technology par bana hai?

Node.js aur Express.js framework par bana hai, jisme RESTful API architecture follow ki gayi hai.

Q2: Passwords kaise secure kiye gaye hain?

Passwords ko plain text mein store karne ke bajaye bcryptjs library se strong salt rounds ke saath hash kiya jata hai.

Q3: User session management kaise hota hai?

Stateless JSON Web Tokens (JWT) ke dwara. Login hone par signed JWT token generate hota hai jo HTTP-Only cookie mein browser ko bheja jata hai.

Q4: Database kounsa hai aur bina install kiye kaise chal raha hai?

Node.js v22.5+ ka naya built-in 'node:sqlite' DatabaseSync engine use kiya gaya hai. Isme kisi external MySQL ya MongoDB server ki zaroorat nahi hoti, ye local file based relational DB hai.

Q5: Render par deploy karte waqt NODE_VERSION=22 kyu zaroori hai?

Kyunki 'node:sqlite' module Node.js version 22 mein introduce hua hai. Older Node 18/20 par ye module nahi hota aur app crash ho sakti hai.